D-test-strategy

D — BobaLink Browser Extension: Complete Test Strategy

Revision: 1 **Last modified:** 2026-06-10T00:00:00Z **Status:** active **Scope:** Defines the 100%-across-all-test-types, anti-bluff testing strategy for the BobaLink MV3 browser extension (the Boba-targeted port of the reference deliverable). **Authority:** CONST §11.4.27 (all test types / 100% type coverage), §11.4.25 (full-automation coverage), §11.4.85 (stress+chaos mandate), §11.4.50 (deterministic consistency), §11.4.107 (no-false-pass / liveness), §11.4.4(b) (four-layer coverage), §11.4 / §11.4.1 (anti-bluff), §11.4.83 (docs/qa evidence), §11.4.98 (full-automation, re-runnable end-to-end).

> Inputs: `_analysis/06-tests.md` (reference suite + bluffs + gaps), `_analysis/01-guides-and-plan.md` (FR/NFR + thresholds), `_analysis/05-src-features.md` (features under test). Repo conventions: `ci.sh`, `tests/{unit,integration,e2e,stress,chaos,load,performance,benchmark,security,property,contract,concurrency,memory,observability}/`, `challenges/scripts/`, `challenges/helixqa-banks/`, `frontend/` (Vitest + Playwright, the Boba runner precedent).

---

0. THE ANTI-BLUFF RULE (the single binding contract)

Every green test MUST prove a user-observable outcome AND fail against a no-op stub of the feature it tests. A test that does not satisfy *both* clauses is a §11.4 PASS-bluff and is a release blocker — worse than a missing test, because it actively misleads.

Operationally, for **every** cell below: 1. The assertion inspects a **user-observable outcome** — a DOM text/attribute the user sees, a `chrome.storage` row, a real HTTP body field, a torrent that actually appears in qBittorrent (`GET /api/v2/torrents/info`), a badge text, a decrypted credential, a rendered popup row — **never** just `status===200`, "no error thrown", or `expect(true).toBe(true)`. 2. It is paired with a **negation check**: the test author confirms (in a `// §1.1 mutation:` comment) that stubbing the production function to a no-op / `return null` / `return []` makes the test FAIL. Where mechanizable, a paired §1.1 mutation gate runs this automatically. 3. **Non-unit tests use real services** (CONST §11.4.27(A)): real Chromium with the real built extension, real `chrome.storage`, real HTTP to 7186/7187/7189. Mocks/stubs/fakes are permitted **only** in `tests/unit/`. Non-unit tests that cannot reach a real service **SKIP-with-reason** (CONST §11.4.3) — never PASS-by-default, never fail-open. 4. Every PASS for a user-visible feature cites a **captured-evidence artifact** under `docs/qa/<run-id>/` (CONST §11.4.83) — screenshot, DOM dump, request/response JSON, the `chrome.storage` snapshot, the qBittorrent `info` row, a latency `.json`, a `result.json`.

---

1. RUNNER DECISION — Vitest (recommended)

Recommendation: standardize the entire extension test stack on Vitest + Playwright. Drop Jest entirely.

Rationale: - **Boba precedent.** `frontend/` already runs **Vitest** (`frontend/vitest.config.ts`, v8 coverage, jsdom) and **Playwright** (`frontend/playwright.config.ts`). One runner across the whole repo = one mental model, one coverage tool, one CI wiring. - **The reference is already broken on this axis.** `crypto.test.ts` imports from `vitest` while the other 6 unit files use Jest globals (06-tests §discrepancy 1). Jest's `testMatch` would pick up the Vitest file and crash on the import. Picking Vitest resolves the inconsistency in the direction the most-thorough file already chose. - **ESM + TS native.** Vitest runs native ESM/TS without `ts-jest`'s `useESM` ceremony — the WXT bundle is ESM. Fewer transform shims. - **jsdom + happy-dom + browser-mode.** Vitest supports jsdom (parser/queue/scanner DOM tests), and `@vitest/browser` (real Chromium) for tests needing real `crypto.subtle`, `MutationObserver` timing fidelity, or real DOM layout — a cleaner escalation path than Jest's jsdom-only world. - **Coverage.** `@vitest/coverage-v8` matches the frontend's existing baseline-just-below-measured gate convention (`COVERAGE_BASELINE.md`), avoiding two coverage toolchains.

Coverage thresholds (override the reference's 80% + UI-excluded model): - The reference `collectCoverageFrom` **excludes** `popup/options/content/background/**` and all `index.ts` (06-tests §discrepancy 7). **This is forbidden** — per CONST §11.4.27, popup, options, content script, and background SW logic **MUST be in coverage**. Move all four INTO the `include` list; keep only `*.d.ts`, `*.css`, `assets/**`, and pure type-only files excluded. - Target **100% across all test types** (the mandate). For line/branch unit coverage, adopt the frontend's ratchet pattern: set the gate just below measured-current and raise toward 100% each cycle (start floor ≥ 90% statements / ≥ 85% branches given the per-file targets the spec already names: crypto 95%, magnet-uri 95%, api-client 90%, queue 85%). Coverage % is necessary but **not** sufficient — the anti-bluff rule (§0) is the real gate.

CI wiring (Boba manual-CI model, CONST Hard-Stop NO CI/CD): there is **no** `.github/workflows`. Strip the reference's `ci.yml` / `release.yml` / Husky hooks / `reporter: 'github'` / `forbidOnly: !!CI` (06-tests §discrepancy 10, 01-guides §K.1). The extension test phases are invoked by an extension-local manual script `extension/ci-ext.sh` (mirrors `./ci.sh`): `lint → typecheck → unit (+coverage gate) → build → integration(live, skip-if-down) → e2e(real extension) → security → perf/load/stress/chaos → challenges → helixqa banks`. `./ci.sh` gains a Phase that shells out to it.

---

2. E2E EXTENSION-LOAD MECHANISM (real extension, real id)

The reference e2e is **non-functional** — hardcoded `chrome-extension://test-id/...`, missing `global-setup.ts`/`global-teardown.ts`, and a Firefox project that can't load the extension via Chromium args (06-tests §discrepancy 4, 5; §"Extension-loading takeaway"). It tests page-DOM shape, not the extension. Remediation — a **real Playwright extension fixture**:

`tests/e2e/fixtures/extension.ts` (Playwright `test.extend`): 1. **Build first.** `playwright.config.ts` `webServer` is replaced by a `globalSetup` that runs `wxt build -b chrome` → produces `.output/chrome-mv3-prod/` (no dev server on :3000 — that was dead config, §discrepancy 4). Assert the manifest + `popup/index.html` + `options/index.html` + content-script bundle exist in the output (CONST §11.4.38 installable-asset evidence — open the artifact, verify each user-visible asset is present and non-degenerate). 2. **Launch persistent context** with the extension loaded (MV3 requires `launchPersistentContext`, not the default browser): `ts const pathToExtension = path.resolve('.output/chrome-mv3-prod'); const context = await chromium.launchPersistentContext(userDataDir, { channel: 'chromium', args: [ `--disable-extensions-except=$`

`{pathToExtension}`,   `--load-extension=${pathToExtension}`,   '--no-first-run',   ],   }); 3. **Resolve the REAL extension id at runtime** (no hardcoded `test-id`): wait for the MV3 service worker target and read its origin — `ts   let [sw] = context.serviceWorkers();   sw ??= await context.waitForEvent('serviceworker');   const extensionId = new URL(sw.url()).host;      // the real assigned id` Expose `extensionId` + a `extensionUrl(path)` helper to every spec. `page.goto(extensionUrl(   'popup/index.html'))` now resolves against the **real loaded extension**. 4. **Teardown** (`globalTeardown` + fixture `context.close()`): close context, delete the temp `userDataDir`, leave no orphan Chromium (CONST §11.4.14 quiescence).

Cross-browser (FR-018, 4 browsers): - **Chromium-family (Chrome/Opera/Yandex)** — all Blink, load identically via `--load-extension`. One Playwright `chromium` project drives Chrome; Opera/Yandex are Chromium-compat → the **adapter + manifest** are validated by the same Chromium e2e plus a build-artifact challenge per target (`wxt build -b opera` etc., assert bundle opens). Full live-browser e2e on Opera/Yandex is `topology_unsupported` SKIP unless those binaries are installed (CONST §11.4.3). - **Firefox 109+ (Gecko, MV3)** — the reference Firefox project is broken (Chromium args don't load extensions in Firefox). Use **`web-ext run`** / `firefox.launchPersistentContext` with the XPI, OR Playwright's Firefox add-on install API, in a **separate Firefox e2e project**. Where the installed Firefox can't be driven headless in CI, SKIP-with-reason + a tracked operator-attended migration item (CONST §11.4.52) — never fake-PASS the Gecko path.

Anti-bluff e2e assertions (must inspect what the user sees, per `frontend/playwright.config.ts` CONST-XII note): not "page loaded" but — popup renders a row whose text === the detected torrent's displayName; clicking Send → the torrent appears in qBittorrent's `GET /api/v2/torrents/info` (`http://localhost:7186`); options "Save Server" → `chrome.storage.local['bobalink_config']` contains the server AND the secret field is ciphertext, not plaintext.

---

3. LIVE-BACKEND INTEGRATION APPROACH (real 7186/7187/7189, skip-if-down)

Integration / e2e / challenge layers hit **real Boba services** (CONST §11.4.27(A) forbids mocks outside unit tests). Per 01-guides §J.3 the spec's fictional 8443/8080 ports **re-map** to:

Spec role	Real Boba service	Port	Extension calls
qBittorrent-direct WebUI `/api/v2/*`	**download-proxy** (injects tracker cookies)	**7186**	`auth/login`, `torrents/add`, `torrents/info`, `app/version`
"Boba Server" REST / search / SSE / dashboard	**merge-search service**	**7187**	search aggregation, dashboard, progress
Search aggregation / Jackett creds	**boba-jackett**	**7189**	indexer overrides, autoconfig

WebUI creds are hardcoded `admin/admin` (CLAUDE.md) — integration tests log in with those against :7186; they **must not** commit any other secret and must verify `boba.db` never holds plaintext.

Reachability gate (CONST §11.4.3 + §11.4.68 `*_require_reachable`): every live test calls a `require_backend(port)` helper at entry: - backend **reachable** → run the real assertion; - backend **unreachable** → `SKIP` with reason `network_unreachable_external` / `feature_disabled_ by_config` (from the §11.4.69 closed reason set) — **never** PASS-by-default, **never** fail-open to a counted SKIP that masks a real failure (the §11.4.68 forbidden pattern). - The Boba `tests/` pattern (`-m "not requires_compose"` in `ci.sh`, `_purge_qbittorrent_torrents` in `tests/stress/`) is the precedent — reuse the same "boot/skip + clean-state" discipline.

Self-driving (CONST §11.4.98): no manual step during a run. `global-setup` may boot the stack via the Containers submodule (`./start.sh` / boot helper, CONST §11.4.76) and seed a known torrent; credential bootstrap (`.env`, `admin/admin`) is the one allowed out-of-band step (§11.4.98(B)). Re-runnable N× with self-cleaning state (purge added torrents before/after, §11.4.50/§11.4.98).

---

4. PERF / LOAD / STRESS THRESHOLDS (from the NFRs — these ARE the gate values)

All from 01-guides §C (NFRs) + §I (edge cases). Each perf/load/stress test asserts the threshold AND captures a latency/throughput artifact (p50/p95/p99) under `docs/qa/<run-id>/` (CONST §11.4.5/ §11.4.85). Thresholds are **calibrated/confirmed**

on Boba's own hardware, not blindly hardcoded (CONST §11.4.107(13)) — the NFR value is the ceiling, the measured baseline ratchets.

Metric	Threshold	NFR	Test type	How measured
Content-script init	≤ 10 ms	NFR-001	performance	`performance.now()` delta on inject
Magnet detection / link	≤ 5 ms	NFR-002	benchmark	benchmark on **1,000-link page**
Popup render	≤ 100 ms	NFR-003	performance	Lighthouse / Playwright trace
Options page load	≤ 200 ms	NFR-004	performance	Lighthouse / trace
API round-trip (local)	≤ 500 ms **p95 over 100 calls**	NFR-005	load + performance	100 live calls to :7186, p95
SW cold start	≤ 50 ms	NFR-006	performance	DevTools performance
Bundle size (compressed)	≤ 350 KB	NFR-007	benchmark + artifact gate	`du -h` on built CRX/ZIP
Detection on 10,000+ links	**no frame > 16 ms** (no UI jank)	§C.4	load + stress	`requestIdleCallback`/ frame timing
Offline queue	1,000 default / **10,000 max**	§C.4, FR-014	scaling + stress	enqueue 10k, assert eviction + persist
Crypto 1 MB encrypt	< 5,000 ms	crypto perf	performance	`performance.now()` around encrypt
Crypto short encrypt / decrypt	< 2,000 ms / < 100 ms	crypto perf	performance	timed roundtrip
Crash-free session	≥ 99.9%	NFR-013	chaos + stress	fault-injection survival rate
Queue durability	100% survive restart	NFR-014	chaos	kill SW / restart, assert 0 loss

Stress floors (CONST §11.4.85 closed-set): sustained ≥ 100 iters OR ≥ 30 s (scan loop, enqueue loop, send loop) with p50/p95/p99 captured; concurrent ≥ 10 parallel (10 tabs scanning, 10 parallel enqueues, 10 parallel sends) — no deadlock, no leak; boundary inputs (0-link page, 10k-link page, empty magnet, 10 MB `.torrent`, off-by-one infohash length). **Load/DDoS-class:** burst of queue enqueues, API request flood against the client rate limiter (10 req/s burst, 60 req/min sustained, FR-025 — assert it throttles, honors 429 `Retry-After`), rapid scan cycles on mutation storms.

---

5. REMEDIATING THE REFERENCE BLUFFS (every one, concretely)

#	Reference bluff (06-tests)	Remediation
B1	`crypto.test.ts` re-implements crypto **inline** and never imports `src/shared/crypto.ts` — cannot fail vs a no-op stub (§11.4.1)	**Import the production `crypto` module** (`encrypt/decrypt/sha256` from `src/shared/crypto.ts`). Keep all 44 conceptual cases (roundtrip, wrong-pw, tamper, IV/salt uniqueness, perf) but exercise the **real** functions. Add §1.1 mutation: stub `decrypt`→returns plaintext → tamper/wrong-pw tests MUST fail. **Also test the real key source** — the reference's fixed passphrase `"bobalink-extension"` and empty-string decrypt (05-src §9.1, §11.4.10) are **security defects the port must fix**; tests assert ciphertext is NOT reversible with a static/empty key and that the chosen real key source roundtrips.
B2	Mixed **Jest + Vitest** runners — `crypto.test.ts` on Vitest, rest on Jest	Standardize on **Vitest** (§1). Rewrite the 6 Jest files to Vitest globals (`vi.fn` for `fetch`/storage mocks). One config, one coverage tool.
B3	**`expect(true).toBe(true)`** no-ops in api-client	**Delete the tautologies; assert**

#	Reference bluff (06-tests)	Remediation
	`Auth` block + queue `Auto-processing` block	**observable state.** `setAuthCookie('sid')` → assert the next request actually sends `Cookie: SID=sid` (spy the real request headers). `setAuthCookie(null)` → assert no Cookie header. Queue auto-processing → assert `processQueue` actually fired (a real send attempt observed against a live/mock server, item state transitioned), not “no throw”.
B4	**Non-functional e2e** — hardcoded `chrome-extension://test-id`, no extension loaded, missing `global-setup`/ `global-teardown`	The **real extension fixture** (§2): build → `launchPersistentContext --load-extension` → resolve real id via service-worker target → drive popup/options/ content against the loaded extension with user-observable assertions. Author the missing `global-setup.ts`/ `global-teardown.ts`.
B5	`setup.ts` declares wrong mock path (`tests/fixtures/chrome-mock`)	Fix to actual path; under Vitest, install the chrome mock via `setupFiles`/ `vi.stubGlobal('chrome', mock)`.
B6	`playwright.config.ts` references missing `global-setup.ts`/`global-teardown.ts` + dead `:3000` webServer + broken Firefox extension project	Author both setup files (build + boot backend + capture id + seed). Remove `:3000` webServer. Fix Firefox to `web-ext`/Firefox-addon load (§2) or SKIP-with-reason.
B7	**Dead-assert** lines: `bencode.test.ts:156` (`.toCharCode ?`	Replace with a real assertion (binary-encode prefix bytes equal expected).

#	Reference bluff (06-tests)	Remediation
	`undefined : undefined`) no-op	
B8	**Coverage excludes UI + entrypoints** (popup/options/ content/background/ index.ts)	Bring all four INTO `collectCoverageFrom/include` (§1). Their logic is now unit + integration tested.
B9	`invalidMagnets` fixture (7 negatives) **never used**; scanner DOM tests use **raw querySelector**, not the production scanner; orchestrator **never runs a scan**	Wire all 7 `invalidMagnets` into magnet tests (each must throw/return null). Replace raw-querySelector DOM tests with calls to the **real `LinkScanner/TextScanner/ScannerOrchestrator`**, asserting detected-torrent output. Add tests that actually run `orchestrator.scanNow()` + MutationObserver-driven rescan + debounce.
B10	api-client test name says `RateLimitError` but asserts `ServerError` (possible inconsistency)	Resolve the contract: assert the **correct** error class the production code throws (verify `handleErrorResponse` — 429 → `RateLimitError` with `retryMs`); fix the test name OR the code to match, with a comment citing the resolution.

---

6. TEST-TYPE × FEATURE MATRIX (compact)

Legend per cell: **what is tested · anti-bluff observable / no-op-stub failure · evidence artifact**. `N/A` cells carry a one-line reason. Columns: U=Unit, I=Integration(live), E=E2E(Playwright real ext), Sec=Security/Pen, L=Load/DDoS, Sc=Scaling, Ch=Chaos, St=Stress, P=Performance, B=Benchmark, UI, UX, C=Challenge, HQ=HelixQA-bank.

6.1 Parser modules (bencode / magnet / .torrent)

Module	U	I	E	Sec
bencode `parser/bencode`	encode/decode 4 types, key-sort, binary, hex, malformed reject · decoded bytes equal expected / stub `decode→{}` fails · `bencode_roundtrip.json`	parser→torrent-file→infohash chain on real `.torrent` · computed infohash matches qBt-reported hash · `infohash_match.json`	N/A — pure logic, no DOM/UI surface	malformed adversaria bencode (d nesting, le overflow, trailing da cannot cra hang · thre bounded, r OOM · `fuzz_repor`
magnet `parser/magnet`	detect/find/dedup, hex+base32 validate+convert, parse(tr/ws/dn/xs/kt), build · parsed infohash lowercased & matches / stub `parseMagnetUri→null` fails · `magnet_parse.json` + **all 7 `invalidMagnets`**	content-script detects real magnet on a live torrent-site page (recorded HTML or live) · detected list contains the magnet's displayName	via content-script e2e (detection on page)	XSS: malic `dn`/magnet injected → escaped in popup DO script exec popup tex escaped, n alert · `xss_magnet`
.torrent `parser/torrent-file`	SHA-1 infohash determinism/ uniqueness/errors, single+multi parse, totalSize, trackers, private, path-join · infohash regex + matches qBt / stub `computeInfohash→''` fails · `torrentfile_parse.json`	parse real downloaded `.torrent` (≤10 MB) → infohash → send → appears in qBt · qBt `info` row hash equals computed · `torrentfile_send.json`	upload `.torrent` via context menu e2e → torrent in qBt	10 MB bou (FR-004 `E_FILE_TOO` 413) enfor corrupt-fil crash

6.2 Network / API / queue layer

Module	U	I	E	Sec
api-client `api/client`	url-normalize, login, version, magnet/file add, GET/POST, retry-5xx, 429 `RateLimitError` (fix B3/	**live :7186**: login(admin/ admin)→add magnet→torrent	send from popup → torrent	HTTPS-on scheme validation (NFR-010)

Module	U	I	E	Sec
	B10), content-type branch, **real Cookie-header propagation** · spied request carries `Cookie: SID=` / stub `requestRaw→{}` fails · `apiclient_unit.json`	in `GET /torrents/info` · qBt state shows the hash · `live_add.json`	appears in qBt (real)	cleartext c logging (NFR-009) scan finds secret · `cred_log_s`
offline-queue `api/queue`	enqueue/dequeue, priority, FIFO eviction, clear, persist, **attempts/backoff/restore-on-init**, processQueue real outcome (fix B3) · storage row written / stub `enqueue`→noop fails; remove `expect(true)` · `queue_unit.json`	queue a send while :7186 down → reconnect → item actually sent → in qBt · qBt shows hash after reconnect · `queue_drain.json`	popup Queue tab: retry/remove/clear change row state	N/A (no ex attack sur beyond sto covered by storage Se
auth `api/auth`	4 methods (none/cookie/api_key/basic), failure-count→`AuthError`, refresh-if-needed, **real `createCredentialsFromConfig` decrypt** · decrypted creds drive a real login / stub `decrypt`→throws caught & fails · `auth_unit.json`	cookie login vs :7186 succeeds; api_key header sent to :7187 · server accepts/rejects observably	options Test-Connection button → real result shown	**credentia rest encry (NFR-008** redaction `Key xxxx`), plaintext i storage/log `chrome.sto` value is ciphertext `cred_at_re`
health `api/health`	thresholds (healthy/degraded/unhealthy), failure-count, testConnection, autoDiscover ports [7186,7187,7189] (re-pointed) · status derived from real latency / stub `getVersion`→’’ fails · `health_unit.json`	**live**: checkServer vs :7187 returns real version + latency · version string matches `app/version` · `live_health.json`	popup status dot reflects real connection (green/orange/red)	N/A

6.3 Detection / scanning / content layer

Module	U	I	E	Sec
scanner: orchestrator `scanner/orchestrator`	**run real scan** (fix B9): scanNow finds seeded magnets; MutationObserver rescan on injected	content-script in real page scans → reports `scan-result` to background → badge updates · badge text	e2e: load page → badge shows N →	stric → sca resp injec case

Module	U	I	E	Sec
	<a>; debounce; dedup · detected count == seeded / stub `performScan`→[] fails · `scan_unit.json`	== count · `scan_integration.json`	popup lists N rows	
scanner: site-db `scanner/site-db`	**all shipped sites** (reconcile `SITE_SELECTORS` vs `SITES`, 05-src §7), known-site recognition, www-match, selector presence, per-site debounce · config.name matches / stub `getSiteConfig`→null fails · `sitedb_unit.json`	content-script uses site selectors on a real/ recorded page from each site · site-specific magnets found	N/A (logic)	priva selec corre (rutr iptor supp
event emitter `shared/events`	on/once/unsub, multi-listener, count, **error isolation** · listener fires once / stub `emit`→noop fails · `events_unit.json`	(exercised via scanner integration)	N/A	N/A
content script `content/index` + highlight	message handlers (scan-now/get-detected/toggle/ status), highlight inject · DOM badge/border/ glow appears / stub injects nothing fails · `content_unit.json` (jsdom)	**real injection** (fix B9): content script runs on a real page, badges appear, message to background observed · highlighted element has `.bobalink-*` class · `content_integration.json`	e2e: highlight visible on page (screenshot diff)	XSS dn in toolt esca `xss_h`

6.4 Background SW / popup / options / cross-cutting

Module	U	I	E
background SW	message router (all 13 handled types),	content→bg→api→qBt full chain; alarms	e2e: context menu Magnet"→qBt; keyb

Module	U	I	E
`background/index`	badge update, context-menu/command/alarm handlers, install seeds DEFAULT_CONFIG · handler returns expected `{success,data}` / stub router→unknown fails · `bg_unit.json`	keep-alive; storage-change re-init · torrent sent end-to-end · `bg_chain.json`	`Ctrl+Shift+B`→send; updates
popup UI `popup/popup`	render rows, selection set, send-enable logic, status dot · row text == displayName / stub render→empty fails · `popup_unit.json` (jsdom)	popup→bg get-detected→render real detected torrents · rows match detected · `popup_integration.json`	**e2e real ext**: empt list renders N detec select→Send enable Send→qBt, status r connection · DOM r qBt row · `popup_e2e.`
options UI `options/options`	form↔config bridge, validation (url), section nav, reset-to-defaults · saved config persisted / stub save→noop fails · `options_unit.json`	**save server →** **`chrome.storage`** **persists + secret encrypted**; test-connection live; auto-discover finds :7186/7187/7189 · storage value is ciphertext · `options_save.json`	**e2e**: add-server modal→save→reloa still there + active; toggle→setting cha reset→defaults · pe state · `options_e2e.`
security / crypto `shared/crypto`	(see B1) AES-256-GCM/PBKDF2 roundtrip, tamper, IV/salt unique, wrong-pw — **on real module** · decrypt fails on tamper / stub decrypt→plaintext fails · `crypto_unit.json`	encrypted cred persisted by options → decrypted by background → drives real login · live login succeeds with decrypted cred · `crypto_chain.json`	N/A (no UI of its ow
cross-browser adapter `BrowserAdapter`	adapter resolves chrome.↔*browser.* per `import.meta.browser`; polyfill present · adapter returns Firefox API on gecko / stub→undefined	N/A (build-time)	e2e per browser pr (Chromium real; Fi real/skip; Opera/Ya artifact)

Module	U	I	E
	fails · `adapter_unit.json`		
tab-groups `TabManager` (net-new, 05-src §4.4)	enumerate groups, send SCAN_REQUEST per tab, aggregate, dedup-across-tabs by infohash · batch list deduped / stub→[] fails · `tabgroup_unit.json`	real: 3 tabs in a group → “Send Tab Group” → all magnets deduped + sent → in qBt · qBt count == unique · `tabgroup_send.json`	e2e: create group, t batch, summary no sent, 0 failed”
i18n `_locales`	all UI strings externalized, `en`+7 locales have every key, no hardcoded text (FR-019) · key lookup returns locale string / stub→key-name fails · `i18n_unit.json`	N/A	e2e: switch locale – options text change
build artifact (manifest/ bundle)	manifest schema valid, MV3, perms minimal, CSP correct, icons rasterized 16/32/48/128 · manifest fields present / missing → fails · `manifest_unit.json`	N/A	N/A

6.5 N/A-cell reasons (DDoS / server-side types on a client extension)

- **DDoS** is **N/A** for pure client-side parsers (no server to flood) — **but the client DOES need Load tests**: 10,000-link-page detection (no jank, §C.4) and API-flood-against-the-client-rate-limiter (FR-025) ARE in-scope and appear in the matrix Load column. “DDoS N/A for a client-side parser; client load = heavy-page + request-burst, covered.”
- **Scaling** is N/A for crypto/events (no scale dimension) but **in-scope** for queue (10k items), detection (10k links), servers list, locales.
- **Security/Pen** is N/A for bencode-internals (covered by parser-fuzz under Sec) — the real attack surface is crypto (at-rest), content-script (XSS via dn), CSP, permissions, credential-leak.

- **UI/UX** are N/A for non-rendering logic modules (parser/api/crypto/events) — those carry no user-facing surface of their own; their UX shows up through popup/options/content cells.

---

7. CHAOS + STRESS DETAIL (CONST §11.4.85 closed-set, per fix-class)

Reuse Boba's `tests/chaos/` + `tests/stress/` + `stress_chaos.sh`-style helpers and the `_purge_qbittorrent_torrents` clean-state pattern. Every PASS cites a captured artifact.

- **Chaos — process-death:** kill the MV3 service worker mid-`processQueue` / mid-send → on restart the queue restores from `chrome.storage.local` with **0 loss** (NFR-014) and resumes. `recovery_ trace.log`.
- **Chaos — network-fault:** drop/delay/reorder responses from :7186 during add/queue → typed `E_NETWORK`/`E_TIMEOUT`, backoff engaged, no hang, item re-queued. `chaos_net.log`.
- **Chaos — input-corruption:** corrupt the persisted queue JSON / a `.torrent` byte / `chrome. storage` config mid-test → `E_STORAGE_CORRUPT`/`E_CRYPTO` handled, recovery to consistent state.
- **Chaos — resource-exhaustion:** `chrome.storage` quota full (`E_STORAGE_FULL`, Firefox stricter, edge case §I) → refuses cleanly, prompts clear-old-items, never crashes.
- **Chaos — state-corruption:** mid-flight config change during a send / partial-write fault → recovery restores consistent config; `cleanup` in `trap`-equivalent leaves quiescent state (§11.4.14).
- **Stress:** sustained scan/enqueue/send ≥ 100 iters or ≥ 30 s with p50/p95/p99; concurrent ≥ 10 (10 tabs, 10 enqueues) — single-resource-owner per tab/sink (§11.4.119), no deadlock/leak; boundary (0/max/off-by-one) inputs categorized.
- Every chaos injection has a **paired §1.1 mutation** (strip the recovery code → the chaos test FAILs), proving the test catches the regression (CONST §11.4.85 4-layer).

---

8. CHALLENGES + HelixQA BANKS + AUTONOMOUS QA (CONST §11.4.27(B), §11.4.98)

Challenges (`challenges/scripts/boba_ext_*.sh`, mirroring existing `cred_roundtrip_challenge.sh`/`credential_leak_grep_challenge.sh` style — build, boot, drive the **real** built extension against **real** :7186/:7187/:7189, assert user-observable outcomes, exit 0/1 with PASS/FAIL message, anti-bluff comment block):

1. `boba_ext_detect_send_magnet_challenge.sh` — load ext → open a real torrent-site page → assert badge count > 0 → popup lists it → Send → `GET :7186/api/v2/torrents/info` contains the hash.
2. `boba_ext_torrentfile_send_challenge.sh` — context-menu "Download .torrent to Boba" (≤10 MB) → torrent in qBt.
3. `boba_ext_offline_queue_durability_challenge.sh` — :7186 down → Send → item queued → restart SW → reconnect → item in qBt (NFR-014 100% durability).
4. `boba_ext_cred_at_rest_challenge.sh` — save server with api-key → assert `chrome.storage.local` value is ciphertext AND no plaintext anywhere AND a static/empty key can't decrypt (fixes B1).
5. `boba_ext_autodiscover_challenge.sh` — auto-discover finds the real :7186/7187/7189 services.
6. `boba_ext_tabgroup_batch_challenge.sh` — 3-tab group → batch send → unique hashes in qBt + "N sent" notification.
7. `boba_ext_artifact_challenge.sh` (CONST §11.4.38) — build each browser target, open the ZIP/CRX, assert manifest + popup html + content bundle + icons + `_locales` present & non-degenerate & bundle ≤ 350 KB.
8. `boba_ext_cred_leak_grep_challenge.sh` — repo-wide grep proves no extension secret is committed (CONST §11.4.10), no fixed passphrase shipped.

HelixQA banks (`challenges/helixqa-banks/boba-ext-*.yaml`, schema per existing `boba-frontend.yaml` — `test_cases` with id/name/category/priority/platforms/steps[action+expected]/ tags/expected_result/fix_reference). One bank per surface: `boba-ext-parser`, `-api`, `-queue`, `-auth`, `-health`, `-scanner`, `-content`, `-background`, `-popup`, `-options`, `-security`, `-tabgroups`, `-i18n`, `-build`, `-xbrowser`. Each case's `action` drives the real extension/backend (Playwright + curl) and `expected` is a user-observable outcome — never "status 200".

Autonomous QA sessions (CONST §11.4.98 / §11.4.50): the banks + challenges run fully self-driving (no human keystroke after start), re-runnable N× with self-cleaning state, scored PASS only on captured evidence. Wired into `extension/ci-ext.sh` and surfaced through the existing `ci.sh` so a single manual run validates the whole extension. Real-time progress via an appendonly JSONL event stream + status snapshot (CONST §11.4.116) so the conductor tails verdicts live, each verdict carrying its evidence path.

---

9. FOUR-LAYER COVERAGE PER FEATURE (CONST §11.4.4(b)) — wiring

Every feature ships **all four**: (1) **pre-build gate** — unit/lint/typecheck + coverage threshold + §1.1 paired-mutation gate (mutate the analyzer/feature → gate FAILs); (2) **post-build** — artifact-open (§11.4.38) asserts the byte landed in the bundle; (3) **runtime-on-clean-target** — e2e/integration/challenge against a freshly-built extension + clean `chrome.storage` (CONST §11.4.108 clean-baseline, no stale profile shadowing the build); (4) **HelixQA Challenge** for every user-visible feature. A fix is "done" only when its declared runtime signature (e.g. "torrent hash appears in qBt `info` after Send") verifies on the clean target — source-green ≠ done (§11.4.108).

---

10. OPEN QUESTIONS FOR THE CONDUCTOR (from 06-tests §"Open questions")

1. **Runner** — confirm Vitest (recommended, §1). 2. **Coverage** — confirm bringing popup/options/ content/background INTO coverage + the floor (≥90/85, ratchet to 100). 3. **Firefox e2e** — real `web-ext` drive vs SKIP-with-reason; is Gecko a release gate? 4. **Crypto key source** — the real replacement for the fixed-`"bobalink-extension"`/empty-string defect (user master passphrase / OS keychain / delegate to boba-jackett's `/config/boba.db`)? Tests must import whatever is chosen.
2. **Backend mapping** — confirm :7186 (qBt-direct via proxy), :7187 (Boba REST/search/SSE), :7189 (boba-jackett) as the live integration targets. 6. **Chrome-mock error-injection** — extend `chrome-mock.ts` with a configurable fault-injecting variant for chaos unit tests (current mock is happy-path only). 7. **Manifest/commands/icons** — authored as part of build-artifact tests.